PostgreSQL Local Environment Guide

Step-by-step database provisioning orchestration and client execution guidelines for VS Code

This reference document maps out the specific execution steps and operational flows required to initialize, interact with, and orchestrate an isolated local PostgreSQL database environment using Docker Compose and Node.js.

Phase 1: Workspace Initialization

1. **Create Project Folder:** Create a new directory on your local device filesystem named `postgres-app`.
2. **Open Workspace:** Launch Visual Studio Code, navigate to the top utility menu, choose `File > Open Folder...`, and target the `postgres-app` directory.
3. **Launch Terminal Panel:** Pull open the VS Code integrated terminal layout framework using the shortkey trigger `Ctrl + ` ` (Windows/Linux) or `Cmd + ` ` (macOS).

Phase 2: Database Container Provisioning (Docker)

1. **Create Configuration Manifest:** In the root level directory space, create a new file explicitly titled `docker-compose.yml` containing the PostgreSQL service schema mapping, database user credentials, database name variables, and standard database communication port `5432`.
2. **Spawning Background Instance:** Start up the container engine stack to provision the background engine using the terminal execution string:

```
docker compose up -d
```

3. **Verification Metric:** Check your environment engine dashboard or type `docker ps` to confirm the engine instance container status shows as completely active and healthy.

Phase 3: Client Infrastructure Initialization

1. **Generate Node Context:** Build out the operational core metadata structural manifest config parameter mappings file via:

```
npm init -y
```

2. **Install PostgreSQL Node Driver:** Add the system client connector library to your system components structure by typing:

```
npm install pg
```

3. **Create Script Entry Point:** Generate a script file named `app.js` in your workspace directory (which will hold your data ingestion schema, dynamic parameters, and query statements).

> **Workspace Checklist:** Confirm your absolute file hierarchy matches the directory map structural architecture down below:
>
> ```
> postgres-app/
> ├── docker-compose.yml
> ├── package.json
> └── app.js
> ```

Phase 4: Runtime Database Operation Pipeline

1. **Dynamic Script Ingestion:** Trigger your programmatic execution sequence while cleanly passing whatever string values you choose directly as custom CLI inputs:
   Example 1 (Basic String Parameter Injection):

   ```
   node app.js "Alice Smith"
   ```

   Example 2 (Alphanumeric Parameter Mapping):

   ```
   node app.js "Developer Admin 01"
   ```

   Example 3 (Fallback Parameter Resolution Check):

   ```
   node app.js
   ```

2. **Observing State Persistence:** Observe the structural grid list output printing out inside your terminal space. Unlike short-lived queue systems, successive execution steps persistently accumulate entries inside your row array storage blocks.

Phase 5: GitHub Copilot Troubleshooting Directives

Leverage the local workspace contexts to fix script problems instantly inside Copilot Chat using these structural patterns:

For Connection Denied Errors (e.g., ECONNREFUSED):

```
@workspace My app.js script cannot establish a socket pipeline on port 5432. Review
docker-compose.yml and verify if my username, password, or mapping definitions
contain structural mismatches.
```

For Schema Mutation Queries:

```
Help me write an updated SQL command blocks matrix for my app.js query section to include a new column tracking user email strings alongside their names.
```

To Wasp-Clean the Persistent State Storage Engine:

To drop all tables and completely purge persistent volume data caches for a pristine environment restart:

```
docker compose down -v
```